

WMConv: Worklist-driven Masked Convolution for Onboard High-Speed Robotic Perception

Anonymous Authors

Abstract—Onboard robotic perception must run convolutional networks within tight latency and power budgets, so many high-speed vision systems produce a spatial active mask and update only the convolution outputs tied to the active regions. Yet existing execution paths for such masked convolution sit at opposite ends of a trade-off between useful-compute ratio and throughput: tile skipping keeps the dense pipeline’s throughput but skips too coarsely, while gather-scatter skips precisely but collapses throughput, so high sparsity rarely becomes a comparable latency speedup. We present WMConv, a worklist-driven dense-grid masked convolution operator that fuses the gather into the convolution kernel and sparsifies only the output positions to compute, retaining the dense pipeline while skipping at position granularity. A lightweight, tiled worklist-construction stage resolves both the coordinate-discovery and locality-preservation overheads at once. On the Jetson Xavier NX and Orin, across three mask sources and three robotic perception tasks, WMConv converts sparsity into on-device latency reduction and higher perception frequency more effectively than dense cuDNN, tile skipping, and gather-scatter baselines, while preserving task accuracy.

Index Terms—Computer architecture for robotic and automation, software, deep learning methods, sparse convolution, embedded perception.

I. INTRODUCTION

ONBOARD robots run convolutional perception in the control loop for navigation, manipulation, and closed-loop control. On embedded platforms the available compute is far below server-class GPUs, yet perception latency is a hard real-time constraint because it sits directly on the control path. Dense convolution recomputes every spatial position, and on these platforms it often cannot reach the update frequency the loop demands. Many high-speed vision systems on UAVs, mobile robots, and embedded devices [1]–[3] therefore concentrate compute on active regions via *masked convolution*: the input stays a dense image-grid tensor, but only the outputs at positions marked active are updated.

The standard dense convolution pipeline partitions the output grid into *tiles* (contiguous spatial blocks) and for each tile reads the required input patch by coordinate, computes the convolution, and writes the result back. This coordinate-driven, tile-based execution achieves high throughput because tiles access spatially contiguous data with regular, predictable memory patterns. Existing masked-convolution systems modify or abandon this pipeline along two paths.

Tile skipping [1], [2] stays inside the dense pipeline: it tests each tile and skips it if every position within is inactive. Throughput is preserved for non-skipped tiles, but a single active position forces the entire tile to execute, so many

computed outputs are unneeded. *Gather-scatter* [3] abandons the pipeline entirely: it extracts active positions exactly, gathers their input neighborhoods, computes, and scatters results back. Every computed output is needed, but the extraction, irregular memory access, and scatter writeback collapse throughput.

We characterize this trade-off along two axes: *throughput* (operations per second, bounded by the dense pipeline) and *useful-compute ratio* (the fraction of executed operations that produce needed outputs). Tile skipping preserves throughput but sacrifices useful-compute ratio; gather-scatter achieves high useful-compute ratio but sacrifices throughput. Neither delivers both, so upstream sparsity does not translate into proportional latency reduction. In our evaluation on Jetson Xavier NX and Orin, existing methods [1]–[3] reduce the active-output ratio to roughly XX–YY%; yet tile-skipping methods achieve only AA–BB% useful-compute ratio, while gather-scatter methods reach only CC% of tile-skipping throughput. Convolution latency is reduced by only DD–EE%.

Our key insight is that the position-level selectivity of gather-scatter can be fused into the dense pipeline’s coordinate-driven execution without the separate data-movement stages. The fusion is straightforward in principle: because each tile already performs a coordinate-driven read-compute-write per position, replacing the tile’s coordinate source, from a contiguous block to a set of active positions, gives position-level selectivity while keeping the per-position execution path identical. This achieves high useful-compute ratio directly, and because the per-position datapath remains that of the dense pipeline, it opens the potential for dense-level throughput.

Realizing this potential requires addressing the challenge of two practical overheads. First, *coordinate discovery*: the kernel must know which positions are active before computation begins, yet scanning for them inside the compute kernel would stall the main datapath. Second, *locality*: if active positions are processed in arbitrary (instead of contiguous) order, memory access loses the spatial regularity that gives the dense pipeline its speed; a global sort would restore order but is itself expensive.

We present WMConv (Worklist-driven Masked Convolution), a dense-grid masked convolution operator that resolves both overheads through a two-stage design. A lightweight construction stage scans the active mask and emits a *worklist*: the ordered coordinates of active output positions. The subsequent compute stage drives the dense pipeline’s coordinate-driven execution with the worklist as its coordinate source, computing only the active outputs. Both overheads are addressed by the construction stage’s tiled scan: it operates entirely in 2D mask space (no feature data touched), so coordinate discovery costs

only XX% of the compute-stage time; and because it proceeds tile by tile over the spatial grid, active positions from the same tile appear contiguously in the worklist, partially restoring the spatial locality of dense execution without any global sort. With construction overhead minimized and only active outputs computed, WMConv achieves near-linear latency scaling: an active-output ratio of $x\%$ yields convolution latency close to $x\%$ of the dense baseline.

This paper makes the following contributions:

- We identify the useful-compute ratio vs. throughput trade-off as the fundamental bottleneck preventing masked convolution from delivering proportional speedup on embedded platforms [1]–[3].
- We present WMConv, a worklist-driven masked convolution operator that fuses position-level selectivity into the dense pipeline’s coordinate-driven execution, achieving both high useful-compute ratio and near-dense throughput.
- On Jetson Xavier NX and Orin, WMConv achieves near-linear latency scaling with active-output ratio across three mask sources and three robotic perception tasks, outperforming dense cuDNN, tile skipping, and gather-scatter while preserving task accuracy.

II. BACKGROUND AND MOTIVATION

A. 2D Dense Convolution on Embedded Platforms

Convolutional neural networks are the computational backbone of onboard robotic visual perception. CNN-based depth estimation and optical flow feed obstacle avoidance and path planning; object detection and semantic segmentation provide scene understanding for navigation; and pose estimation enables closed-loop manipulation. While vision transformers have achieved strong accuracy on these tasks, their quadratic attention cost makes them impractical under the tight latency and power budgets of embedded platforms; CNN-based models remain the dominant deployed choice for real-time onboard visual inference. Across these models, 2D convolutional layers operating on image-grid feature maps dominate the inference workload and sit directly on the control-critical path.

On embedded platforms (Jetson Xavier NX, Orin, edge TPUs), power and thermal budgets limit available compute to a fraction of server-class GPUs, yet the closed control loop imposes a hard latency ceiling: perception must complete within the sensor frame interval (typically tens of milliseconds) to feed downstream planning and control in time.

A 2D convolution layer transforms an input feature map $\mathbf{x} \in \mathbb{R}^{H_{in} \times W_{in} \times C_{in}}$ into an output $\mathbf{y} \in \mathbb{R}^{H_{out} \times W_{out} \times C_{out}}$ using a weight tensor $\mathbf{W} \in \mathbb{R}^{C_{out} \times C_{in} \times k \times k}$, where k is the kernel size and $k \times k$ is the receptive field size. The computation is defined per position on the output grid $\mathcal{G}_{out} = \{1, \dots, H_{out}\} \times \{1, \dots, W_{out}\}$. For each $p \in \mathcal{G}_{out}$, let $\mathcal{N}(p) \subset \mathcal{G}_{in}$ denote the set of k^2 input positions in the receptive field of p . The layer computes:

$$\mathbf{y}[p] = \sum_{i=1}^{k^2} \mathbf{W}_i \mathbf{x}[\mathcal{N}_i(p)], \quad (1)$$

where $\mathcal{N}_i(p)$ is the i -th position in the receptive field and $\mathbf{W}_i \in \mathbb{R}^{C_{out} \times C_{in}}$ is the corresponding weight slice. The

output at each position is thus an accumulation of k^2 weighted contributions from the input. On the GPU, $\mathcal{G}_{out}$ is partitioned into N disjoint tiles $\mathcal{G}_{out} = T_1 \cup \dots \cup T_N$, $T_i \cap T_j = \emptyset$ for $i \neq j$, where each tile T_j is a contiguous spatial block of output positions. Tiles are processed in parallel; within each tile, positions are spatially adjacent, so their receptive fields overlap and lie in contiguous memory. This regularity enables cuDNN-class [4] throughput. We denote by $C \subseteq \mathcal{G}_{out}$ the set of positions actually computed by a given execution strategy. Dense convolution sets $C = \mathcal{G}_{out}$ unconditionally, which on embedded platforms often exceeds the latency budget.

B. Masked Convolution for Onboard Perception

1) *Mask Sources*: Many onboard vision systems exploit the fact that only a subset of the input changes between inference calls or is relevant to the current decision. Let $\mathcal{G}_{in} = \{1, \dots, H_{in}\} \times \{1, \dots, W_{in}\}$ denote the input spatial grid. These systems identify an input active mask $M_{in} \subseteq \mathcal{G}_{in}$ marking positions where the input has changed or is task-relevant. An output position p must be recomputed whenever its receptive field intersects the input mask, giving the output active mask:

$$M = \{p \in \mathcal{G}_{out} \mid \mathcal{N}(p) \cap M_{in} \neq \emptyset\}. \quad (2)$$

In a multi-layer network, the output mask of layer l becomes the input mask of layer $l+1$, propagating sparsity through the model. Three families of mask sources produce the initial M_{in} :

a) *Event-camera increments.*: Event cameras are bio-inspired sensors in which each pixel independently fires when its brightness change exceeds a threshold, generating data only where intensity varies. They offer microsecond temporal resolution, high dynamic range, and negligible motion blur, making them widely used on UAVs and high-speed platforms. Events accumulated over a temporal window are differenced to reveal which positions received new activity, producing a naturally sparse mask.

b) *Temporal frame differences.*: Standard frame-based cameras on mobile robots and autonomous vehicles capture video at a fixed rate. Between consecutive frames most of the scene remains static, especially under moderate ego-motion. Temporal differencing methods [2] compare consecutive frames (or motion-compensated histories) and identify positions where pixel differences exceed a threshold; only outputs affected by these positions need recomputation.

c) *Learned spatial gating.*: Rather than relying on temporal change, dynamic networks [3] train a lightweight gating module that examines the current input and predicts, per spatial region, whether computation is needed for the downstream task. Regions classified as low-value are masked out, adapting the compute budget to task-relevant content per frame.

2) *Execution Paradigms*: Given a mask M , the system must compute $\mathbf{y}[p]$ only for $p \in M$. Existing methods follow one of two paradigms:

a) *Tile skipping.*: A tile T_j is executed if and only if $T_j \cap M \neq \emptyset$; otherwise it is skipped entirely. The computed set is $C = \bigcup_{j: T_j \cap M \neq \emptyset} T_j$. Because non-skipped tiles run the unmodified dense pipeline, throughput is preserved. However,

Fig. 1. Active ratio vs. actual convolution speedup on Jetson Orin for the three mask sources. The dashed line marks ideal (linear) scaling. Tile skipping and gather-scatter both fall far short.

$|C|$ can be much larger than $|M|$ when active positions are scattered across many tiles: a single active position forces its entire tile to execute. Ev-Conv [1] and DeltaCNN [2] follow this paradigm.

b) Gather-scatter: Active positions are extracted exactly: the system gathers $\{x[\mathcal{N}(p)] : p \in M\}$ into a contiguous buffer, computes Eq. (1) for each, and scatters results back to the output grid. The computed set is $C = M$ exactly, so no computation is wasted. However, the gather, scatter, and irregular memory access patterns abandon the dense pipeline’s regular execution, reducing throughput to a fraction of tile-based methods. DGNet [3] follows this paradigm.

C. The Gap: Sparsity Does Not Become Speedup

For a network with L convolution layers, we define the *active ratio* $\rho = \frac{1}{L} \sum_{l=1}^L |M^l|/|\mathcal{G}_{out}^l|$, the average fraction of output positions that require computation across all layers. We characterize masked-convolution efficiency along two axes: *throughput*, the rate of convolution operations per second (bounded by the dense pipeline), and *useful-compute ratio*, the fraction of executed operations that produce needed outputs. Tile skipping preserves throughput but sacrifices useful-compute ratio; gather-scatter achieves high useful-compute ratio but sacrifices throughput. Neither delivers both, so upstream sparsity does not translate into proportional latency reduction.

Figure 1 quantifies the problem on Jetson Orin. Across the three mask sources, the active ratio ρ reaches XX–YY%, yet tile-skipping methods achieve only AA–BB% useful-compute ratio (limited by tile granularity) and gather-scatter methods reach only CC% of tile-skipping throughput (limited by data-movement overhead). As a result, convolution latency is reduced by only DD–EE%, far below the ideal linear scaling where $\rho = x\%$ would yield $x\%$ of dense latency. This gap motivates WMConv.

D. 3D Sparse Convolution: A Different Problem

3D sparse convolution (SpConv v2 [?], TorchSparse++ [?]) targets point-cloud workloads where input data is inherently irregular: active voxels occupy scattered positions with no underlying dense grid. The gathered features have no spatial regularity, so the core challenge is how to impose computational structure on irregular data. Implicit GEMM, the dominant approach, fuses the per-offset gather into a single matrix multiplication with index mapping, achieving structured computation despite irregular access.

WMConv faces a fundamentally different problem. The input data resides on a regular dense grid; reading any position’s neighborhood is a structured, stride-based access. The challenge is not how to compute efficiently on irregular data, but how to skip inactive outputs without leaving the dense pipeline’s already-efficient execution.

Applying 3D sparse convolution directly to this 2D dense-grid scenario performs poorly in practice. These systems require building coordinate hash tables and neighbor maps that involve mandatory GPU synchronization, equivalent in cost to their “full convolution” path. In our scenario the active mask changes every frame (or every event window), so this construction cannot be amortized across layers as in 3D SubM convolution. On Jetson Orin, SpConv v2 applied to a 2D masked convolution with $\rho = XX\%$ runs $XX\times$ slower than the dense baseline due to construction overhead alone (Section ??).

ACKNOWLEDGMENTS

REFERENCES

- [1] S. Durvasula, Y. Guan, and N. Vijaykumar, “Ev-conv: Fast CNN inference on event camera inputs for high-speed robot perception,” *IEEE Robotics Autom. Lett.*, vol. 8, no. 6, pp. 3174–3181, 2023. [Online]. Available: <https://doi.org/10.1109/LRA.2023.3259731>
- [2] M. Parger, C. Tang, C. D. Twigg, C. Keskin, R. Wang, and M. Steinberger, “Deltacnn: End-to-end CNN inference of sparse frame differences in videos,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2022, New Orleans, LA, USA, June 18-24, 2022*. IEEE, 2022, pp. 12 487–12 496. [Online]. Available: <https://doi.org/10.1109/CVPR52688.2022.01217>
- [3] F. Li, G. Li, X. He, and J. Cheng, “Dynamic dual gating neural networks,” in *2021 IEEE/CVF International Conference on Computer Vision, ICCV 2021, Montreal, QC, Canada, October 10-17, 2021*. IEEE, 2021, pp. 5310–5319. [Online]. Available: <https://doi.org/10.1109/ICCV48922.2021.00528>
- [4] S. Chetlur, C. Woolley, P. Vandermersch, J. Cohen, J. Tran, B. Catanzaro, and E. Shelhamer, “cudnn: Efficient primitives for deep learning,” *CoRR*, vol. abs/1410.0759, 2014. [Online]. Available: <http://arxiv.org/abs/1410.0759>